TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

BE Programme | Year III / Part II

EMBEDDED SYSTEM (CT 655)

QUESTION BANK WITH ANSWERS

Chapter 10: VHDL

COURSE SYLLABUS AT A GLANCE

Unit	Topic	Hours	Marks
1	Introduction to Embedded System	3	4
2	Hardware Design Issues	4	8
3	Software Design Issues	6	8
4	Memory	5	8
5	Interfacing	6	8
6	Real-Time Operating System (RTOS)	8	12
7	Control System	3	8
8	IC Technology	3	8
9	Microcontrollers in Embedded Systems	3	8
10	VHDL	4	8
	Total	**45**	**80**

Chapter 10 Topics (4 Hours | 8 Marks):

- 10.1 VHDL Overview — Entity, Architecture, Library, Signal, Process
- 10.2 VHDL Architecture Styles — Behavioral, Dataflow, Structural
- 10.3 Finite State Machine (FSM) Design with VHDL
- 10.4 Combinational Circuits — Full Adder, MUX, Decoder
- 10.5 Sequential Circuits — D FF, JK FF, T FF, Registers
- 10.6 Counters — 4-bit Up Counter, 3-bit Up Counter (Structural)
- 10.7 Custom Processors — GCD, LCM, Factorial (Algorithm + FSM + VHDL)
- 10.8 Traffic Light Controller, Sequence Detector (FSM-based)

Chapter 10: VHDL

Q1. Explain different architecture models/styles of VHDL with examples. What are the basic components of a VHDL design?	2079 Chaitra (Regular) | 2078 Poush (Back) | 2080 Ashwin (Back) [2+6]

ANSWER

VHDL Design Structure:

Every VHDL design has two mandatory parts: the ENTITY (interface — defines port names and directions) and the ARCHITECTURE (implementation — defines behavior/structure).

```
-- Basic VHDL design skeleton:
library IEEE;                           -- library declaration
use IEEE.STD_LOGIC_1164.ALL;            -- package for std_logic types
use IEEE.NUMERIC_STD.ALL;               -- package for arithmetic

entity ENTITY_NAME is                   -- INTERFACE definition
  Port(                                 -- port list
    input_a  : in  STD_LOGIC;           -- input port
    input_b  : in  STD_LOGIC_VECTOR(3 downto 0);
    output_y : out STD_LOGIC;           -- output port
    clk      : in  STD_LOGIC            -- clock
  );
end ENTITY_NAME;

architecture ARCH_NAME of ENTITY_NAME is
  -- Signal declarations (internal wires)
  signal internal_sig : STD_LOGIC;
begin
  -- Architecture body: concurrent statements / processes
end ARCH_NAME;
```

Three VHDL Architecture Styles:

1. Behavioral Model: Describes WHAT the circuit does (its algorithm/function) using PROCESS statements, sequential if/case/loop statements. The synthesis tool decides the gate-level implementation. Highest abstraction level.

```
-- Behavioral: AND gate
architecture Behavioral of AND_gate is
begin
  process(A, B)         -- sensitivity list: re-runs when A or B changes
  begin
    if A='1' and B='1' then
      Y <= '1';
    else
```

```
      Y <= '0';
    end if;
  end process;
end Behavioral;
```

2. Dataflow Model: Describes circuit using concurrent signal assignments and Boolean expressions. Signals are assigned using <= continuously (not inside a process). Mid-level abstraction.

```
-- Dataflow: Full Adder
architecture Dataflow of Full_Adder is
begin
  Sum  <= A XOR B XOR Cin;                        -- concurrent
  Cout <= (A AND B) OR (B AND Cin) OR (A AND Cin); -- concurrent
end Dataflow;
-- Both statements execute simultaneously (not sequentially)
```

3. Structural Model: Describes circuit by instantiating and connecting sub-components — equivalent to a schematic netlist. Explicitly connects component ports via signals. Lowest abstraction (closest to gate level).

```
-- Structural: 2-bit AND using component instantiation
architecture Structural of Top is
  component AND2                  -- component declaration
    Port(A, B : in STD_LOGIC; Y : out STD_LOGIC);
  end component;
  signal w1 : STD_LOGIC;         -- internal wire
begin
  U1: AND2 port map(A=>In1, B=>In2, Y=>w1);  -- instance U1
  U2: AND2 port map(A=>w1,  B=>In3, Y=>Out1); -- instance U2
end Structural;
```

Style	Abstraction	Key Constructs	Best Used For
Behavioral	Highest	PROCESS, if/elsif, case, loop	FSMs, algorithms, complex sequential logic
Dataflow	Medium	Concurrent signal assignments, Boolean operators	Combinational logic, simple expressions
Structural	Lowest	COMPONENT, PORT MAP, signals as wires	Hierarchical designs, netlists, reuse of blocks

Q2. Write VHDL code for a Full Adder using Dataflow style. Then implement a 4-bit Binary Parallel Adder (BPA) using Structural style.	**2081 Ashwin (Back) | 2073 Magh (Back) | 2075 Bhadra (Back) [2+6]**

ANSWER

Full Adder Truth Table:

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

```
-- ==========================================================
-- FULL ADDER — Dataflow Style
-- Sum  = A XOR B XOR Cin
-- Cout = (A AND B) OR (B AND Cin) OR (A AND Cin)
-- ==========================================================
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Full_Adder is
  Port(
    A, B, Cin : in  STD_LOGIC;
    Sum, Cout : out STD_LOGIC
  );
end Full_Adder;

architecture Dataflow of Full_Adder is
begin
  Sum  <= A XOR B XOR Cin;
  Cout <= (A AND B) OR (B AND Cin) OR (A AND Cin);
end Dataflow;

-- ==========================================================
-- 4-BIT BINARY PARALLEL ADDER — Structural Style
-- Ripple Carry: carry output of FA_i feeds cin of FA_(i+1)
-- ==========================================================
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity BPA_4bit is
  Port(
    A   : in  STD_LOGIC_VECTOR(3 downto 0);   -- 4-bit input A
    B   : in  STD_LOGIC_VECTOR(3 downto 0);   -- 4-bit input B
    Cin : in  STD_LOGIC;                       -- carry in
```

```
    Sum : out STD_LOGIC_VECTOR(3 downto 0);   -- 4-bit sum
    Cout: out STD_LOGIC                        -- carry out
  );
end BPA_4bit;

architecture Structural of BPA_4bit is

  -- Component declaration (reuse Full_Adder entity)
  component Full_Adder
    Port(A, B, Cin : in STD_LOGIC; Sum, Cout : out STD_LOGIC);
  end component;

  -- Internal carry signals between stages
  signal c1, c2, c3 : STD_LOGIC;

begin
  -- Bit 0 (LSB): carry in = external Cin
  FA0: Full_Adder port map(A=>A(0), B=>B(0), Cin=>Cin, Sum=>Sum(0),
Cout=>c1);

  -- Bit 1: carry in = c1 from FA0
  FA1: Full_Adder port map(A=>A(1), B=>B(1), Cin=>c1,  Sum=>Sum(1),
Cout=>c2);

  -- Bit 2: carry in = c2 from FA1
  FA2: Full_Adder port map(A=>A(2), B=>B(2), Cin=>c2,  Sum=>Sum(2),
Cout=>c3);

  -- Bit 3 (MSB): carry out = final Cout
  FA3: Full_Adder port map(A=>A(3), B=>B(3), Cin=>c3,  Sum=>Sum(3),
Cout=>Cout);

end Structural;
```

Q3. What is behavioral modeling? Write VHDL code for a 2-input multiplexer (MUX). Also write VHDL for a 4-to-1 MUX.	**2072 Magh (Back) | 2070 Magh (Back) | 2076 Baisakh (Back) [2+6]**

ANSWER

Behavioral Modeling:

Behavioral modeling in VHDL describes the functionality of a circuit at a high algorithmic level using PROCESS statements and sequential constructs (if/elsif/case/loop). It does not specify how the circuit is physically built — the synthesis tool maps it to gates. It is the most commonly used style for designing FSMs, controllers, and complex sequential circuits.

- Key characteristics: Uses PROCESS with sensitivity list; all statements inside PROCESS are sequential (like software); outside PROCESS all statements are concurrent.
- Advantage: Easy to write and understand; closely resembles pseudocode; fast simulation.
- Disadvantage: Less direct control over hardware implementation compared to structural style.

```
-- ==========================================================
-- 2-to-1 MULTIPLEXER — Behavioral Style
-- Y = D0 when sel='0';  Y = D1 when sel='1'
-- ==========================================================
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity MUX2to1 is
  Port(
    D0, D1 : in  STD_LOGIC;   -- data inputs
    sel    : in  STD_LOGIC;   -- select: 0=D0, 1=D1
    Y      : out STD_LOGIC    -- output
  );
end MUX2to1;

architecture Behavioral of MUX2to1 is
begin
  process(D0, D1, sel)          -- sensitivity list
  begin
    if sel = '0' then
      Y <= D0;                  -- select input D0
    else
      Y <= D1;                  -- select input D1
    end if;
  end process;
end Behavioral;

-- ==========================================================
-- 4-to-1 MULTIPLEXER — Behavioral Style (using CASE)
-- Y = D(sel) based on 2-bit select
-- ==========================================================
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity MUX4to1 is
  Port(
    D   : in  STD_LOGIC_VECTOR(3 downto 0);  -- 4 data inputs
    sel : in  STD_LOGIC_VECTOR(1 downto 0);  -- 2-bit select
    Y   : out STD_LOGIC                       -- output
  );
```

```
end MUX4to1;

architecture Behavioral of MUX4to1 is
begin
  process(D, sel)
  begin
    case sel is
      when "00" => Y <= D(0);   -- sel=00: output D0
      when "01" => Y <= D(1);   -- sel=01: output D1
      when "10" => Y <= D(2);   -- sel=10: output D2
      when "11" => Y <= D(3);   -- sel=11: output D3
      when others => Y <= '0';
    end case;
  end process;
end Behavioral;
```

Q4. What is structural style of architecture in VHDL? Write VHDL code for a JK flip-flop and a D flip-flop with synchronous reset.	**2081 Chaitra (Regular) | 2079 Jestha (Back) | 2075 Baisakh (Back) [3+5]**

ANSWER

Structural Style in VHDL:

Structural modeling describes a circuit as an interconnection of component instances, similar to a schematic netlist. It uses COMPONENT declarations to define the sub-module interface, then PORT MAP statements to instantiate and wire them together using internal signals.

- Key constructs: COMPONENT declaration (defines the component interface), PORT MAP (instantiates the component and connects its ports to signals), internal SIGNAL declarations (act as wires).
- Advantage: Maps directly to hardware; supports hierarchical design (build complex systems from verified sub-blocks); easy to reuse components.
- Disadvantage: More verbose; does not describe function directly; less flexible for algorithmic descriptions.

```
-- ═══════════════════════════════════════════════════════
-- JK FLIP-FLOP — Behavioral Style
-- J=0,K=0: No change (hold)  | J=0,K=1: Reset (Q=0)
-- J=1,K=0: Set   (Q=1)       | J=1,K=1: Toggle (Q=NOT Q)
-- ═══════════════════════════════════════════════════════
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity JK_FlipFlop is
  Port(
    clk   : in  STD_LOGIC;
    reset : in  STD_LOGIC;    -- asynchronous active-HIGH reset
    J, K  : in  STD_LOGIC;
```

```
    Q     : out STD_LOGIC;
    Qbar  : out STD_LOGIC
  );
end JK_FlipFlop;

architecture Behavioral of JK_FlipFlop is
  signal Q_int : STD_LOGIC := '0';  -- internal state
begin
  process(clk, reset)
  begin
    if reset = '1' then             -- async reset
      Q_int <= '0';
    elsif rising_edge(clk) then     -- update on rising clock edge
      case (J & K) is               -- concatenate J and K
        when "00" => Q_int <= Q_int;      -- Hold
        when "01" => Q_int <= '0';         -- Reset
        when "10" => Q_int <= '1';         -- Set
        when "11" => Q_int <= not Q_int;   -- Toggle
        when others => Q_int <= Q_int;
      end case;
    end if;
  end process;
  Q    <= Q_int;
  Qbar <= not Q_int;
end Behavioral;

-- ═══════════════════════════════════════════════════════
-- D FLIP-FLOP — with synchronous reset
-- On rising clock edge: if reset=1 → Q=0; else Q=D
-- ═══════════════════════════════════════════════════════
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity D_FlipFlop is
  Port(
    clk   : in  STD_LOGIC;
    reset : in  STD_LOGIC;    -- synchronous active-HIGH reset
    D     : in  STD_LOGIC;
    Q     : out STD_LOGIC;
    Qbar  : out STD_LOGIC
  );
end D_FlipFlop;

architecture Behavioral of D_FlipFlop is
  signal Q_int : STD_LOGIC := '0';
begin
  process(clk)
  begin
    if rising_edge(clk) then
```

```
      if reset = '1' then     -- synchronous reset (checked on clock
edge)
        Q_int <= '0';
      else
        Q_int <= D;           -- capture D on rising edge
      end if;
    end if;
  end process;
  Q    <= Q_int;
  Qbar <= not Q_int;
end Behavioral;
```

J	K	Q(next)	Operation
0	0	Q (no change)	Hold current state
0	1	0	Reset
1	0	1	Set
1	1	NOT Q	Toggle

Q5. Design a 3-bit synchronous up counter using T flip-flops in VHDL structural style. Show the state diagram and excitation table.

2079 Ashwin (Back) | 2076 Chaitra (Back) | 2075 Bhadra (Back) [8]

ANSWER

3-bit Up Counter — State Diagram:

```
States: 000 → 001 → 010 → 011 → 100 → 101 → 110 → 111 → 000 (wraps)

T Flip-Flop excitation: T = Q(next) XOR Q(current)
For counter: Q2Q1Q0 counts 0-7 then wraps

Excitation table for 3-bit up counter:
┌──────────────┬──────────────┬──────────────┐
│ CurrentQ2Q1Q0│ Next Q2Q1Q0  │  T2  T1  T0  │
├──────────────┼──────────────┼──────────────┤
│    0 0 0     │   0 0 1      │   0   0   1  │
│    0 0 1     │   0 1 0      │   0   1   1  │
│    0 1 0     │   0 1 1      │   0   0   1  │
│    0 1 1     │   1 0 0      │   1   1   1  │
│    1 0 0     │   1 0 1      │   0   0   1  │
│    1 0 1     │   1 1 0      │   0   1   1  │
│    1 1 0     │   1 1 1      │   0   0   1  │
│    1 1 1     │   0 0 0      │   1   1   1  │
└──────────────┴──────────────┴──────────────┘
Pattern: T0=1 always; T1=Q0; T2=Q1 AND Q0
```

```
-- ═══════════════════════════════════════════════════════════
-- T FLIP-FLOP (building block for counter)
-- On rising_edge: if T=1 → toggle Q; if T=0 → hold Q
-- ═══════════════════════════════════════════════════════════
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity T_FlipFlop is
  Port(
    clk   : in  STD_LOGIC;
    reset : in  STD_LOGIC;
    T     : in  STD_LOGIC;
    Q     : out STD_LOGIC;
    Qbar  : out STD_LOGIC
  );
end T_FlipFlop;

architecture Behavioral of T_FlipFlop is
  signal Q_int : STD_LOGIC := '0';
begin
  process(clk, reset)
  begin
    if reset = '1' then
      Q_int <= '0';
    elsif rising_edge(clk) then
      if T = '1' then
        Q_int <= not Q_int;   -- toggle
      end if;                 -- T=0: hold (no change)
    end if;
  end process;
  Q    <= Q_int;
  Qbar <= not Q_int;
end Behavioral;

-- ═══════════════════════════════════════════════════════════
-- 3-BIT SYNCHRONOUS UP COUNTER — Structural Style
-- T0=1 (always toggle), T1=Q0, T2=Q1 AND Q0
-- ═══════════════════════════════════════════════════════════
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Counter_3bit is
  Port(
    clk   : in  STD_LOGIC;
    reset : in  STD_LOGIC;
    Q     : out STD_LOGIC_VECTOR(2 downto 0)  -- Q(2)=MSB, Q(0)=LSB
  );
end Counter_3bit;
```

```
architecture Structural of Counter_3bit is

  component T_FlipFlop
    Port(clk, reset, T : in STD_LOGIC; Q, Qbar : out STD_LOGIC);
  end component;

  signal Q0, Q1, Q2       : STD_LOGIC;  -- FF outputs
  signal Q0b, Q1b, Q2b    : STD_LOGIC;  -- FF complemented outputs
  signal T0, T1, T2       : STD_LOGIC;  -- T inputs to each FF

begin
  -- T excitation equations derived from truth table:
  T0 <= '1';              -- T0 is always 1 (bit 0 always toggles)
  T1 <= Q0;               -- T1 = Q0 (bit 1 toggles when Q0=1)
  T2 <= Q1 AND Q0;        -- T2 = Q1·Q0 (bit 2 toggles when Q1=Q0=1)

  -- Instantiate 3 T flip-flops
  FF0: T_FlipFlop port map(clk=>clk, reset=>reset, T=>T0, Q=>Q0,
Qbar=>Q0b);
  FF1: T_FlipFlop port map(clk=>clk, reset=>reset, T=>T1, Q=>Q1,
Qbar=>Q1b);
  FF2: T_FlipFlop port map(clk=>clk, reset=>reset, T=>T2, Q=>Q2,
Qbar=>Q2b);

  -- Connect outputs
  Q(0) <= Q0;
  Q(1) <= Q1;
  Q(2) <= Q2;

end Structural;
```

Q6. Write VHDL code for a 4-bit synchronous up counter with synchronous reset and enable using behavioral style.

Model Question | 2077 Chaitra (Back) | 2074 Bhadra (Back) [4+4]

ANSWER

```
-- ═══════════════════════════════════════════════════════
-- 4-BIT SYNCHRONOUS UP COUNTER — Behavioral Style
-- Counts 0000 to 1111 (0 to 15), then wraps to 0000
-- Synchronous reset: count resets to 0 on next clock edge
-- Enable: counter only counts when en='1'
-- ═══════════════════════════════════════════════════════
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;         -- for UNSIGNED type and arithmetic

entity Counter_4bit is
```

```
  Port(
    clk   : in  STD_LOGIC;
    reset : in  STD_LOGIC;        -- synchronous active-HIGH reset
    en    : in  STD_LOGIC;        -- count enable: 1=count, 0=hold
    Q     : out STD_LOGIC_VECTOR(3 downto 0);  -- 4-bit count output
    TC    : out STD_LOGIC         -- Terminal Count: 1 when Q=1111
  );
end Counter_4bit;

architecture Behavioral of Counter_4bit is
  signal count : UNSIGNED(3 downto 0) := (others => '0');
begin

  process(clk)
  begin
    if rising_edge(clk) then
      if reset = '1' then
        count <= (others => '0');  -- synchronous reset to 0000
      elsif en = '1' then
        count <= count + 1;        -- increment (wraps 1111→0000 auto)
      end if;                      -- en=0: hold current count
    end if;
  end process;

  Q  <= STD_LOGIC_VECTOR(count);        -- convert UNSIGNED to
STD_LOGIC_VECTOR
  TC <= '1' when count = "1111" else '0'; -- terminal count flag

end Behavioral;

-- ─── Timing diagram (rising edge triggered) ───
--  clk:   _|‾|_|‾|_|‾|_|‾|_|‾|_|‾|_|‾|_|‾|_|‾|_
--  reset: ‾‾‾‾‾‾‾‾|_____________________________
--  en:    ‾‾‾‾‾‾‾‾‾‾‾‾‾‾|___|‾‾‾‾‾‾‾‾‾‾‾‾‾‾‾‾
--  Q:      0  0  0  0   1   1  2  3  4  5 ...
--              ↑reset        ↑hold ↑continue
```

Q7. Write a VHDL code to design a traffic light controller using a finite state machine (FSM). State your assumptions clearly.

2079 Chaitra (Regular) | 2077 Baisakh (Back) | 2074 Jestha (Back) [2+6]

ANSWER

Assumptions:

- Clock period = 1 second (or a timer is used to count clock cycles for each state duration).
- 3 light states: RED (30 clock cycles), GREEN (25 clock cycles), YELLOW (5 clock cycles).

- Output: lights(2)=RED, lights(1)=YELLOW, lights(0)=GREEN. Only one light ON at a time.
- Active HIGH output: '1' = light ON.

FSM State Diagram:

```
         [counter=30]           [counter=25]          [counter=5]
  ┌──────────────────────────────────────────────────────────────┐
  │                                                              │
  ▼                                                              │
RED_S ──(count=30)──▶ GREEN_S ──(count=25)──▶ YELLOW_S ──(count=5)──┘
(lights=100)          (lights=001)             (lights=010)

Reset → RED_S (counter=0)
```

```
-- ═══════════════════════════════════════════════════════
-- TRAFFIC LIGHT CONTROLLER — FSM (Behavioral VHDL)
-- States: RED_S (30 cycles), GREEN_S (25 cycles), YELLOW_S (5 cycles)
-- ═══════════════════════════════════════════════════════
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Traffic_Light is
  Port(
    clk    : in  STD_LOGIC;
    reset  : in  STD_LOGIC;
    lights : out STD_LOGIC_VECTOR(2 downto 0)  -- (2)=RED (1)=YEL (0)=GRN
  );
end Traffic_Light;

architecture Behavioral of Traffic_Light is

  -- FSM state type
  type state_type is (RED_S, GREEN_S, YELLOW_S);
  signal state   : state_type := RED_S;
  signal counter : INTEGER range 0 to 50 := 0;

begin
  process(clk, reset)
  begin
    if reset = '1' then
      state   <= RED_S;
      counter <= 0;

    elsif rising_edge(clk) then
      counter <= counter + 1;    -- increment every clock cycle
```

```
      case state is

        when RED_S =>
          lights <= "100";        -- RED ON
          if counter = 30 then    -- stay RED for 30 cycles
            state   <= GREEN_S;
            counter <= 0;
          end if;

        when GREEN_S =>
          lights <= "001";        -- GREEN ON
          if counter = 25 then    -- stay GREEN for 25 cycles
            state   <= YELLOW_S;
            counter <= 0;
          end if;

        when YELLOW_S =>
          lights <= "010";        -- YELLOW ON
          if counter = 5 then     -- stay YELLOW for 5 cycles
            state   <= RED_S;
            counter <= 0;
          end if;

      end case;
    end if;
  end process;
end Behavioral;
```

Q8. Design a VHDL sequence detector for the bit sequence '1011' using a Mealy FSM. Draw the state diagram.	**2078 Chaitra (Back) | 2076 Bhadra (Back) [3+5]**

ANSWER

Sequence Detector for '1011' — Mealy FSM:

A Mealy FSM output depends on both the current state AND the current input. The detector outputs detected='1' for one clock when the last 4 bits received = '1011'.

```
State Diagram (Mealy — output shown on transition arc as input/output):

         0/0          0/0
   ┌──── S0 ──────────────────────────────────────────────┐
   │   (init) ◀──────────────────────── 0/0 ──  S3 ◀──────┤
   │     │                              1/1 ──▶ S0        │
   │     │ 1/0                                            │
   │     ▼                                                 |
```

```
   |    S1 ── (got '1') ── 0/0 ──▶ S2 ── 1/0 ──▶ S3          |
   |     |  ◀──────────── 1/0 ── S1      |                   |
   |     |                               | 1/1 (got 1011!)   |
   |     | 1/0 (self-loop)               ▼                   |
   └─────┘                              OUT=1                |
                                                             |
States:                                                      |
  S0: No relevant bits received yet (initial state)          |
  S1: Received '1'                                           |
  S2: Received '10'                                          |
  S3: Received '101'  → waiting for final '1'                |
  On S3 + input='1': output=1 (sequence detected!), go S0    |
```

```
-- ═══════════════════════════════════════════════════════
-- SEQUENCE DETECTOR for '1011' — Mealy FSM
-- Inputs:  clk, reset, x (serial input bit)
-- Output:  detected ('1' for one cycle when 1011 seen)
-- ═══════════════════════════════════════════════════════
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Seq_Detector is
  Port(
    clk      : in  STD_LOGIC;
    reset    : in  STD_LOGIC;
    x        : in  STD_LOGIC;   -- serial input
    detected : out STD_LOGIC    -- '1' when 1011 detected
  );
end Seq_Detector;

architecture Behavioral of Seq_Detector is
  type state_type is (S0, S1, S2, S3);
  signal state : state_type := S0;
begin

  -- State register (sequential process)
  process(clk, reset)
  begin
    if reset = '1' then
      state <= S0;
    elsif rising_edge(clk) then
      case state is

        when S0 =>                 -- initial: waiting for '1'
          if x = '1' then state <= S1;   -- got first '1'
          else             state <= S0;   -- got '0', stay
          end if;
```

```
        when S1 =>                  -- received '1'
          if x = '0' then state <= S2;   -- got '10'
          else             state <= S1;   -- got '11', keep S1 (last '1')
          end if;

        when S2 =>                  -- received '10'
          if x = '1' then state <= S3;   -- got '101'
          else             state <= S0;   -- got '100', reset
          end if;

        when S3 =>                  -- received '101'
          if x = '1' then state <= S1;   -- got '1011'! → back to S1
          else             state <= S2;   -- got '1010', last '10' saved
          end if;

      end case;
    end if;
  end process;

  -- Mealy output: combinational (depends on state AND current input x)
  detected <= '1' when (state = S3 and x = '1') else '0';

end Behavioral;
```

Q9. Write an algorithm and VHDL code for a custom processor that calculates the Greatest Common Divisor (GCD) of two numbers. Draw the FSM state diagram.

2082 Kartik (Back) | 2073 Bhadra (Back) | 2071 Bhadra (Regular) [3+5]

ANSWER

Algorithm for GCD (Euclidean Subtraction Method):

1. Load inputs A and B into registers Ra and Rb.
2. Compare Ra and Rb.
3. If Ra = Rb: GCD found → output Ra → done.
4. If Ra > Rb: Ra = Ra − Rb → go to step 2.
5. If Rb > Ra: Rb = Rb − Ra → go to step 2.

FSM State Diagram:

```
  reset                go=1                Ra=Rb
   |                    |                    |
   ▼                    ▼                    ▼
S_IDLE ──(go=1)──▶ S_COMPARE ──(Ra=Rb)──▶ S_DONE ──▶ S_IDLE
                        |                            (result<=Ra, done<=1)
                        | Ra>Rb
```

```
                        ▼
                     S_SUB_A
                    (Ra<=Ra-Rb)
                        │
                        └──────────────────────────► S_COMPARE
                        │ Ra<Rb (else)
                        ▼
                     S_SUB_B
                    (Rb<=Rb-Ra)
                        │
                        └──────────────────────────► S_COMPARE
```

```
-- ═══════════════════════════════════════════════════════
-- GCD PROCESSOR — Behavioral VHDL (FSM implementation)
-- Algorithm: Euclidean subtraction
-- Inputs: a_in, b_in (8-bit), go (start pulse), clk, reset
-- Outputs: result (GCD), done (completion flag)
-- ═══════════════════════════════════════════════════════
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity GCD_Processor is
  Port(
    clk    : in  STD_LOGIC;
    reset  : in  STD_LOGIC;
    go     : in  STD_LOGIC;                    -- start computation
    a_in   : in  UNSIGNED(7 downto 0);          -- first number
    b_in   : in  UNSIGNED(7 downto 0);          -- second number
    result : out UNSIGNED(7 downto 0);          -- GCD output
    done   : out STD_LOGIC                      -- '1' when complete
  );
end GCD_Processor;

architecture Behavioral of GCD_Processor is
  type state_type is (S_IDLE, S_COMPARE, S_SUB_A, S_SUB_B, S_DONE);
  signal state   : state_type := S_IDLE;
  signal Ra, Rb  : UNSIGNED(7 downto 0);
begin

  process(clk, reset)
  begin
    if reset = '1' then
      state  <= S_IDLE;
      done   <= '0';
      Ra     <= (others => '0');
      Rb     <= (others => '0');
```

```
    elsif rising_edge(clk) then
      case state is

        when S_IDLE =>
          done <= '0';
          if go = '1' then
            Ra    <= a_in;           -- load inputs
            Rb    <= b_in;
            state <= S_COMPARE;
          end if;

        when S_COMPARE =>
          if Ra = Rb then
            state <= S_DONE;         -- GCD found
          elsif Ra > Rb then
            state <= S_SUB_A;        -- subtract Rb from Ra
          else
            state <= S_SUB_B;        -- subtract Ra from Rb
          end if;

        when S_SUB_A =>
          Ra    <= Ra - Rb;          -- Ra = Ra - Rb
          state <= S_COMPARE;

        when S_SUB_B =>
          Rb    <= Rb - Ra;          -- Rb = Rb - Ra
          state <= S_COMPARE;

        when S_DONE =>
          result <= Ra;              -- Ra = Rb = GCD
          done   <= '1';
          state  <= S_IDLE;          -- ready for next computation

      end case;
    end if;
  end process;
end Behavioral;

-- Trace example: GCD(12, 8)
--  Ra=12, Rb=8 → Ra>Rb → Ra=12-8=4
--  Ra=4,  Rb=8 → Rb>Ra → Rb=8-4=4
--  Ra=4,  Rb=4 → Ra=Rb → GCD=4 ✓
```

Q10. Write an algorithm and VHDL code for a custom processor that calculates the Least Common Multiple (LCM) of two numbers as a finite state machine.	**2074 Bhadra (Regular) | 2072 Ashwin (Regular) | 2078 Chaitra (Back) [3+5]**

ANSWER

Algorithm for LCM:

1. Save original values: oa = a, ob = b.
2. Compute product = a × b (saved for later).
3. Find GCD using Euclidean subtraction: repeatedly subtract smaller from larger until Ra = Rb.
4. LCM = product / GCD = (a × b) / GCD.
5. Output LCM, assert done.

```
-- ═══════════════════════════════════════════════════
-- LCM PROCESSOR — Behavioral VHDL
-- LCM(a,b) = (a × b) / GCD(a,b)
-- ═══════════════════════════════════════════════════
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity LCM_Processor is
  Port(
    clk    : in  STD_LOGIC;
    reset  : in  STD_LOGIC;
    go     : in  STD_LOGIC;
    a_in   : in  UNSIGNED(7 downto 0);
    b_in   : in  UNSIGNED(7 downto 0);
    result : out UNSIGNED(15 downto 0);   -- LCM (up to 16 bits)
    done   : out STD_LOGIC
  );
end LCM_Processor;

architecture Behavioral of LCM_Processor is
  type state_type is (IDLE, COMPARE, SUB_A, SUB_B, COMPUTE_LCM, DONE_S);
  signal state   : state_type := IDLE;
  signal Ra, Rb  : UNSIGNED(7 downto 0);
  signal product : UNSIGNED(15 downto 0);  -- a*b stored before GCD
begin

  process(clk, reset)
  begin
    if reset = '1' then
      state <= IDLE;  done <= '0';

    elsif rising_edge(clk) then
      case state is

        when IDLE =>
          done <= '0';
```

```
          if go = '1' then
            Ra      <= a_in;
            Rb      <= b_in;
            product <= a_in * b_in;  -- save product = a*b
            state   <= COMPARE;
          end if;

        when COMPARE =>
          if Ra = Rb then     state <= COMPUTE_LCM;
          elsif Ra > Rb then  state <= SUB_A;
          else                state <= SUB_B;
          end if;

        when SUB_A =>
          Ra <= Ra - Rb;  state <= COMPARE;

        when SUB_B =>
          Rb <= Rb - Ra;  state <= COMPARE;

        when COMPUTE_LCM =>
          -- Ra now holds GCD(a,b); LCM = product / GCD
          result <= product / Ra;
          state  <= DONE_S;

        when DONE_S =>
          done  <= '1';
          state <= IDLE;

      end case;
    end if;
  end process;
end Behavioral;

-- Trace example: LCM(6, 4)
--  product = 6×4 = 24
--  GCD(6,4): 6>4→Ra=2; 2<4→Rb=2; Ra=Rb=2 → GCD=2
--  LCM = 24/2 = 12 ✓
```

Q11. Write an algorithm and VHDL code for a custom processor that calculates the factorial of n (n!) as a finite state machine.	**2073 Magh (Back) | 2081 Ashwin (Back) | 2075 Bhadra (Back) [2+6]**

ANSWER

Algorithm for Factorial (n!):

1. Load n into counter register R_cnt. Initialize result register R_res = 1.
2. Check: if R_cnt ≤ 1 → factorial done (0!=1, 1!=1). Jump to DONE.

3. Multiply: R_res = R_res × R_cnt.
4. Decrement: R_cnt = R_cnt − 1.
5. Go back to step 2.
6. Output R_res = n!, assert done.

FSM State Diagram:

```
  reset                  go=1
   |                      |
   ▼                      ▼
 IDLE ──(go=1)──▶ INIT ──────▶ CHECK ──(R_cnt≤1)──▶ DONE_S
                  (R_res=1,    |                    (result<=R_res
                   R_cnt=n)    | R_cnt>1             done<=1)
                               ▼
                          MULTIPLY ──▶ DECREMENT ──▶ CHECK
                         (R_res=       (R_cnt=
                          R_res×cnt)    R_cnt-1)
```

```
-- ═══════════════════════════════════════════════════════
-- FACTORIAL PROCESSOR — Behavioral VHDL
-- Computes n! for n = 0 to 8 (8! = 40320 fits in 16 bits)
-- ═══════════════════════════════════════════════════════
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Factorial is
  Port(
    clk    : in  STD_LOGIC;
    reset  : in  STD_LOGIC;
    go     : in  STD_LOGIC;
    n_in   : in  UNSIGNED(3 downto 0);    -- n (0-8 for 16-bit result)
    result : out UNSIGNED(15 downto 0);   -- n! result
    done   : out STD_LOGIC
  );
end Factorial;

architecture Behavioral of Factorial is
  type state_t is (IDLE, INIT, CHECK, MULTIPLY, DECREMENT, DONE_S);
  signal state : state_t := IDLE;
  signal R_res : UNSIGNED(15 downto 0);
  signal R_cnt : UNSIGNED(3 downto 0);
begin

  process(clk, reset)
  begin
    if reset = '1' then
```

```
      state <= IDLE;  done <= '0';

    elsif rising_edge(clk) then
      case state is

        when IDLE =>
          done <= '0';
          if go = '1' then state <= INIT; end if;

        when INIT =>
          R_res <= x"0001";   -- result = 1
          R_cnt <= n_in;       -- counter = n
          state <= CHECK;

        when CHECK =>
          if R_cnt <= 1 then   -- 0! = 1! = 1, done
            state <= DONE_S;
          else
            state <= MULTIPLY;
          end if;

        when MULTIPLY =>
          R_res <= R_res * R_cnt;   -- result = result * counter
          state <= DECREMENT;

        when DECREMENT =>
          R_cnt <= R_cnt - 1;       -- counter--
          state <= CHECK;

        when DONE_S =>
          result <= R_res;
          done   <= '1';
          state  <= IDLE;

        when others => state <= IDLE;
      end case;
    end if;
  end process;
end Behavioral;

-- Trace example: 4! = 24
--  INIT: R_res=1, R_cnt=4
--  MULTIPLY: R_res=1*4=4, DECREMENT: R_cnt=3
--  MULTIPLY: R_res=4*3=12, DECREMENT: R_cnt=2
--  MULTIPLY: R_res=12*2=24, DECREMENT: R_cnt=1
--  CHECK: R_cnt=1 ≤ 1 → DONE: result=24 ✓
```

Summary: Topics Covered in Chapter 10

S.N.	Topic / VHDL Code	Exam(s) Asked	Marks
1	VHDL Overview — entity/architecture structure, port modes, library	2079 Chaitra, 2078 Poush, 2080 Ashwin	2–4
2	Behavioral vs Dataflow vs Structural — comparison table + code examples for each	2079 Chaitra, 2078 Poush (annual)	2+6
3	Full Adder — Dataflow style (XOR/AND equations)	2073 Magh, 2075 Bhadra, 2075 Baisakh (annual)	3+5
4	4-bit Binary Parallel Adder (BPA) — Structural style (component + port map)	2081 Ashwin, 2073 Magh (annual)	2+6
5	2-to-1 MUX — Behavioral style (if/else)	2072 Magh, 2070 Magh (annual)	2+6
6	4-to-1 MUX — Behavioral style (case statement)	2072 Magh, Model Q	4+4
7	JK Flip-Flop — Behavioral style (case on J&K, toggle/set/reset/hold)	2081 Chaitra, 2078 Poush, 2075 Baisakh (annual)	3+5
8	D Flip-Flop — with synchronous reset (rising_edge + if/else)	2079 Jestha, 2075 Baisakh	4+4
9	3-bit Up Counter — Structural style (T FFs + excitation table + T equations)	2079 Ashwin, 2076 Chaitra, 2075 Bhadra (annual)	8
10	4-bit Up Counter — Behavioral style (UNSIGNED + enable + terminal count)	2077 Chaitra, 2074 Bhadra, Model Q	4+4
11	Traffic Light Controller — FSM (3 states, timer counter, state diagram)	2079 Chaitra, 2077 Baisakh, 2074 Jestha (annual)	2+6
12	Sequence Detector '1011' — Mealy FSM (4 states, state diagram + VHDL)	2078 Chaitra, 2076 Bhadra (annual)	3+5
13	GCD Processor — Algorithm (Euclidean subtraction) + FSM diagram + VHDL	2082 Kartik, 2073 Bhadra, 2071 Bhadra (annual)	3+5
14	LCM Processor — Algorithm (product/GCD) + FSM + VHDL	2074 Bhadra, 2072 Ashwin, 2078 Chaitra (annual)	3+5
15	Factorial Processor — Algorithm (loop multiply) + FSM diagram + VHDL	2073 Magh, 2081 Ashwin, 2075 Bhadra (annual)	2+6